

DATA236 - Distributed Systems for Data Engineering

HOMEWORK 4

Total Point: 30

Instructions:

- Include screenshots of the code for each component, as well as the code for app.js, along with the corresponding output. Ensure that the code and its output are displayed together, one below the other.
- Submission should be in PDF Format.
- Please name your submission file as {last_name}_HW4.pdf

Q1. React (10 Points)

Create a Book Management App where users can add, update, and delete books.

Add a Login page/component (Login.jsx) that allows a user to login using email + password.

- After successful login, the backend will set an HTTP-only cookie session (session_id).
- Only logged-in users should be allowed to view the book list and perform CRUD operations.
- If the user is not logged in:
 - show a message “Login required”, and/or
 - disable/hide navigation buttons like “Add Book”.

I. Home Page (1 Point)

Write the necessary code to create the ‘Home.jsx’ component which will display all the books. The Home component should be rendered when the user is on the root route ‘/’.

II. Add a New Book (2 Points)

Write the necessary code to create the ‘CreateBook.jsx’ component. The component should be rendered when the user is on the ‘/create’ route.

The component should accept props to add the new book and have the following:

- An input field for entering the Book Title.
- An input field for entering the Author Name.
- A submit button labeled "Add Book".

When the user clicks the Add book button:

- A new book should be added to the book list with an auto-incremented book ID.
- The user should be redirected to the home page to see the updated book list.

III. Update Book (2 Points)

Write the necessary code to create the 'UpdateBook.jsx' component. The component should be rendered when the user is on the '/update' route.

The component should accept props to update the new book and have the following:

- An input field for Book Title.
- An input field for Author Name.
- A submit button labeled "Update Book".

When the user clicks the Update Book button the book within context should be updated and the user should be redirected to the home page showing the updated book list.

IV. Delete the Book (2 Points)

Write the necessary code to create the 'DeleteBook.jsx' component. The component should be rendered when the user is on the '/delete' route.

The component should accept props (similar to demo) to delete a book and have the following:

- A button labeled "Delete Book".

When the user clicks the Delete Book button the book in context should be deleted and the user should be redirected to the home page showing the updated book list.

V. For all the above questions make sure to:

- Pass props for the Create, Update, and Delete components. **(1 Point)**
- Use hooks like useState and useEffect wherever necessary. **(1 Point)**
- Use React-Router-Dom for routing. **(1 Point)**

Q2. MySQL (10 points)

In this question, you will apply the concepts learned in class to build a simple CRUD (Create, Read, Update, Delete) application using FastAPI and MySQL using SQLAlchemy

Database

Create MySQL database: book_db

Table: books (auto-increment id, title, author)

Also create tables for authentication/session management:

- users (id, name, email (unique), password_hash)
- sessions (id (session token), user_id, created_at, expires_at)

You are required to create a Book Management System that allows users to:

- Add a new book (POST)
- View all books (GET)

- View a book by ID (GET)
- Update book details (PUT)
- Delete a book (DELETE)

You are free to name your API endpoints appropriately.

Submit the Postman API response screenshots for each operation. Each completed operation will get 2 points. Also, submit a screenshot of the database and project folder structure

Part 2: Compare Three LlamaIndex Chunking Techniques (Retrieval-Only RAG) 21 points

Goal

Implement three chunking strategies in LlamaIndex using the Tiny Shakespeare dataset, build in-memory vector indexes, run retrieval-only queries, and compare retrieval quality with simple metrics + short written observations.

Chunking techniques to implement

1. Token-based chunking — `TokenTextSplitter`
2. Semantic chunking — `SemanticSplitterNodeParser`
3. Sentence-window chunking — `SentenceWindowNodeParser`

Dataset

Use this raw text file:

Tiny Shakespeare:

<https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt>

Environment & Setup

Install:

- `llama-index`
- `llama-index-embeddings-huggingface`
- `sentence-transformers`
- `faiss-cpu`
- `numpy`
- `Pandas`

Embedding model:

- Use a public sentence embedding model from Hugging Face (example):
 - `sentence-transformers/all-MiniLM-L6-v2`

What you must build

1) One retrieval-only pipeline per technique

For each chunker (**Token** / **Semantic** / **Sentence-window**), do:

A. Chunking

- **TokenTextSplitter**: choose sensible `chunk_size` and `chunk_overlap`.
- **SemanticSplitterNodeParser**: choose `buffer_size`; use the **same embed model** to find semantically coherent boundaries.
- **SentenceWindowNodeParser**: split into sentences and attach a context window (neighbor sentences) in metadata so surrounding context is available.

B. Indexing (in-memory)

- Build a `VectorStoreIndex` using an **in-memory vector store** (e.g., `SimpleVectorStore`) to keep everything local and fast.

2) Retrieval-only helper function (required)

Write a helper function:

Input: `query`, `k`, and the technique's retriever/index

Output: printed diagnostics + a ranked table

For a given query:

1. Compute the query embedding
 - Print: embedding dimension and first 8 values
2. Retrieve top-k nodes
3. For each retrieved node, compute and print:
 - store similarity score (if your retriever exposes it)
 - cosine similarity between query embedding and the chunk embedding (explicitly compute embeddings of returned chunks)
 - chunk length (characters is fine)
 - preview (first ~160 characters)
4. Print:
 - query vector shape
 - stacked document vectors shape

Your printed output must clearly label the technique and show a table:

rank	store_score	cosine_sim	chunk_len	preview
------	-------------	------------	-----------	---------

Query (must use for all three)

Use this query for all pipelines:

- Query: *Who are the two feuding houses?*

Optional (to strengthen your comparison), add 1–2 more queries such as:

- “Who is Romeo in love with?”
- “Which play contains the line ‘To be, or not to be’?”

Comparison metrics (report section)

After running all three pipelines, report for each technique:

1. Retrieval quality
 - `top-1 cosine` (highest cosine among top-k)
 - `mean@k cosine` (average cosine over top-k)
2. Chunking stats
 - `#chunks` produced
 - `avg chunk length` (characters or tokens)
3. Latency
 - retrieval latency in milliseconds (simple timer around the similarity search)

Write-up requirements

Observations (1–2 short paragraphs)

Explain why one method performed better for the required query. Discuss factors like:

- coherence of chunks
- boundary quality (semantic splitting)
- token budget alignment
- context preserved via sentence window metadata

If results differ across optional extra queries, mention it.

Conclusion (2–5 sentences)

State which technique you judge best for this corpus and why.

Submission (PDF)

Submit **one PDF** that includes:

- your code (notebook export is fine)
- your outputs (printed tables/diagnostics)
- the comparison metrics + observations + conclusion

Evaluation Rubric (21 points)

1) Environment & Setup — 2 pts

- 1 pt: Required packages installed/imported successfully
- 1 pt: Uses a public Hugging Face sentence embedding model (e.g., `all-MiniLM-L6-v2`) and shows it’s wired into the pipelines

2) Data Loading & Preprocessing — 2 pts

- 1 pt: Loads Tiny Shakespeare from the provided URL (or downloaded file) correctly
- 1 pt: Basic cleaning/handling shown (e.g., reading as one document, sanity check like length/preview)

3) Chunking Implementations (3 techniques) — 6 pts

- 2 pts: Token-based chunking implemented with sensible `chunk_size` + `chunk_overlap`
- 2 pts: Semantic chunking implemented with `SemanticSplitterNodeParser` using the embedding model, with a chosen `buffer_size`
- 2 pts: Sentence-window chunking implemented with `SentenceWindowNodeParser`, includes neighbor-window metadata/context

4) In-Memory Indexing (per technique) — 3 pts

- 1 pt: Builds `VectorStoreIndex` for all three techniques
- 1 pt: Uses an in-memory store (e.g., `SimpleVectorStore`) consistently (no external DB)
- 1 pt: Indexing is correct and retriever can be constructed for each technique

5) Retrieval-Only Helper Function & Required Printouts — 4 pts

- 1 pt: Prints query embedding dimension + first 8 values
- 1 pt: Retrieves top-k nodes and prints a clear ranked table with: `rank`, `store_score`, `cosine_sim`, `chunk_len`, `preview`
- 1 pt: Explicitly computes chunk embeddings for returned chunks and cosine similarity (not just store score)
- 1 pt: Prints vector shapes (query vector + stacked doc vectors)

6) Metrics & Timing Comparison — 2 pts

- 1 pt: Reports `top-1 cosine` and `mean@k cosine` per technique
- 1 pt: Reports `#chunks`, `avg chunk length`, and retrieval latency (ms) per technique

7) Discussion & Conclusion — 2 pt

- 1 pt: Observations (1–2 paragraphs) tie differences to chunk coherence/boundaries/context
- 1 pt: Conclusion (2–5 sentences) states which technique is best for this corpus and why

Eval